

Sistemas Distribuidos y Programación Paralela

# Trabajo Práctico 1

## Conceptos básicos de Sistemas Distribuidos

Informe de cierre — versión final

### Grupo Cerberus

Salvador Baez (195157) · Mateo Nomico (168102) · Tomás Resnik (190168)

Universidad Nacional de Luján · Departamento de Ciencias Básicas

Cátedra: Dr. David Petrocelli · Comisión Mercedes · Segundo cuatrimestre de 2026

Lenguaje: Python 3.11+ · Repositorio: [github.com/mnomico/sdypp\\_tp1](https://github.com/mnomico/sdypp_tp1) · Tag: v1.1

Septiembre de 2026 · revisado en octubre de 2026

# Contenido

1. Presentación y alcance de esta versión
2. Qué se construyó, hit por hit
3. Arquitectura final
4. Métricas y gráficas: JSON sobre TCP frente a gRPC con Protobuf
5. Pruebas
6. Integración continua y despliegue público
7. Seguridad
8. Herramientas de IA: qué usamos y en qué ayudaron
9. Conclusiones
10. Video y anexo: cómo ejecutar

# 1. Presentación y alcance de esta versión

El TP1 pide construir, en ocho pasos incrementales ("hits"), un sistema de nodos que se saludan por la red: primero un cliente y un servidor TCP, después un programa único que es cliente y servidor a la vez, luego un registro de contactos que descubre a los demás nodos, un sistema de inscripciones por ventanas de tiempo y, por último, la migración de la comunicación a gRPC con Protocol Buffers. A eso se suman los requisitos transversales de la materia: repositorio público con un README por hit, pruebas, health-checks, logs, pipeline de CI/CD, despliegue público, seguridad sin secrets en el código, video e informe.

La primera entrega resolvió los ocho hits con 77 pruebas en verde. Este documento es el informe de cierre de la versión final, que completa los cuatro puntos pendientes de la devolución de la cátedra:

- **Despliegue público.** El sistema corre en una máquina virtual de AWS EC2 con el /health del Nodo D accesible desde Internet. La VM trae por su cuenta cada imagen que publica el pipeline, sin ningún secret guardado en GitHub ni en la VM (sección 6).
- **Gráficas en el informe.** La comparativa JSON vs Protobuf del Hit #8 se volvió a medir con 1000 llamadas y se presenta con tres gráficas (sección 4).
- **Herramientas de IA.** Se detalla qué asistente usó cada integrante y para qué (sección 8).
- **Video.** Un video por hit, con el sistema corriendo, en el README de cada carpeta (sección 10).

El detalle técnico de cada hit (diagrama de arquitectura, instrucciones de ejecución y decisiones de diseño) está en el README de su carpeta en el repositorio; este informe los resume y agrega lo que atraviesa a todos: métricas, pruebas, despliegue y conclusiones.

**Revisión de octubre de 2026 (v1.1).** Corrige la sección 6 y la Figura 5: la versión anterior describía un despliegue en Google Cloud Run, que fue el plan original y no llegó a hacerse. Lo que corre es una VM de AWS EC2 que trae la imagen desde el registro de GitHub. Con eso se corrigen también la autenticación del despliegue en la sección 7, el puntero al video en la sección 10 y la cantidad de pruebas (82).

## 2. Qué se construyó, hit por hit

Cada hit es una carpeta autocontenida (código, pruebas y README) y reutiliza el módulo común (sección 3). La tabla resume la consigna y cómo se resolvió; todas están completas.

| Hit | Consigna                                                                                | Cómo se resolvió                                                                                                                                                                                                                                                |
|-----|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| #1  | Servidor TCP B que espera y responde el saludo de A; cliente A que se conecta y saluda. | Framing por '\n' sobre el flujo TCP (LectorDeMensajes acumula hasta el delimitador). B atiende una conexión y termina limpio; lógica separada del main para probarla con puerto efímero.                                                                        |
| #2  | A reconecta y resaluda si B cierra la conexión (p. ej. kill -9).                        | Backoff exponencial con tope (0,5 s → 5 s) que se reinicia tras cada conexión exitosa. A queda bloqueado leyendo tras saludar: así detecta la caída. Un RST (kill -9) y un cierre ordenado siguen el mismo camino de reintento.                                 |
| #3  | B sigue funcionando si A muere.                                                         | Un hilo por conexión; toda excepción del cliente (cierre, RST, bytes ilegibles, mensaje demasiado largo) termina sólo ese hilo y queda en el log. Timeout de inactividad de 60 s por canal. Primer endpoint HTTP /health, en la misma interfaz que el servicio. |
| #4  | Programa único C, cliente y servidor a la vez; dos instancias cruzadas se saludan.      | Un canal por sentido (saliente con el backoff del Hit #2, entrante con el accept() del Hit #3). El orden de arranque es indiferente. El canal saliente nunca muere: se marca degradado y sigue reintentando.                                                    |

| Hit | Consigna                                                                                               | Cómo se resolvió                                                                                                                                                                                                                                                  |
|-----|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| #5  | Los mensajes viajan en JSON.                                                                           | JSON Lines (un objeto por línea) con version, id, tipo, origen, contenido, timestamp y en_respuesta_a. Se valida al deserializar y lo inválido se cuenta en el /health sin cortar el canal. Serialización separada del transporte: mensajes.py no conoce sockets. |
| #6  | Nodo D como registro de contactos; C escucha en puerto aleatorio, se registra en D y recibe los pares. | D guarda los nodos en un arreglo en RAM protegido por un Lock y responde con los otros nodos. C hace bind al puerto 0 y publica el que le asignó el SO: N instancias en una misma máquina sin coordinar puertos. /health con cantidad de nodos, uptime y estado.  |
| #7  | Inscripciones por ventanas fijas de 1 minuto; los C sólo ven la ventana actual; persistencia en JSON.  | Dos listas en RAM (activos / siguientes) y un hilo que cada 60 s promueve una a la otra. D devuelve únicamente los nodos de la ventana activa. Escritura atómica del archivo JSON (tmp + replace) para que nunca quede a medias.                                  |
| #8  | Reemplazar JSON sobre TCP por gRPC con Protocol Buffers y comparar tamaño, latencia y experiencia.     | Contrato en servicio.proto (servicios MensajeríaC y RegistroD), stubs generados con protoc, llamadas unarias Saludar / Registrar / ConsultarActivos. Soporta par fijo y registro dinámico. Sin tocar comun/: los Hits #1-#7 siguen intactos.                      |

### 3. Arquitectura final

El sistema creció por acumulación: cada hit conservó lo que ya funcionaba y agregó una capacidad. La figura resume las cuatro etapas.

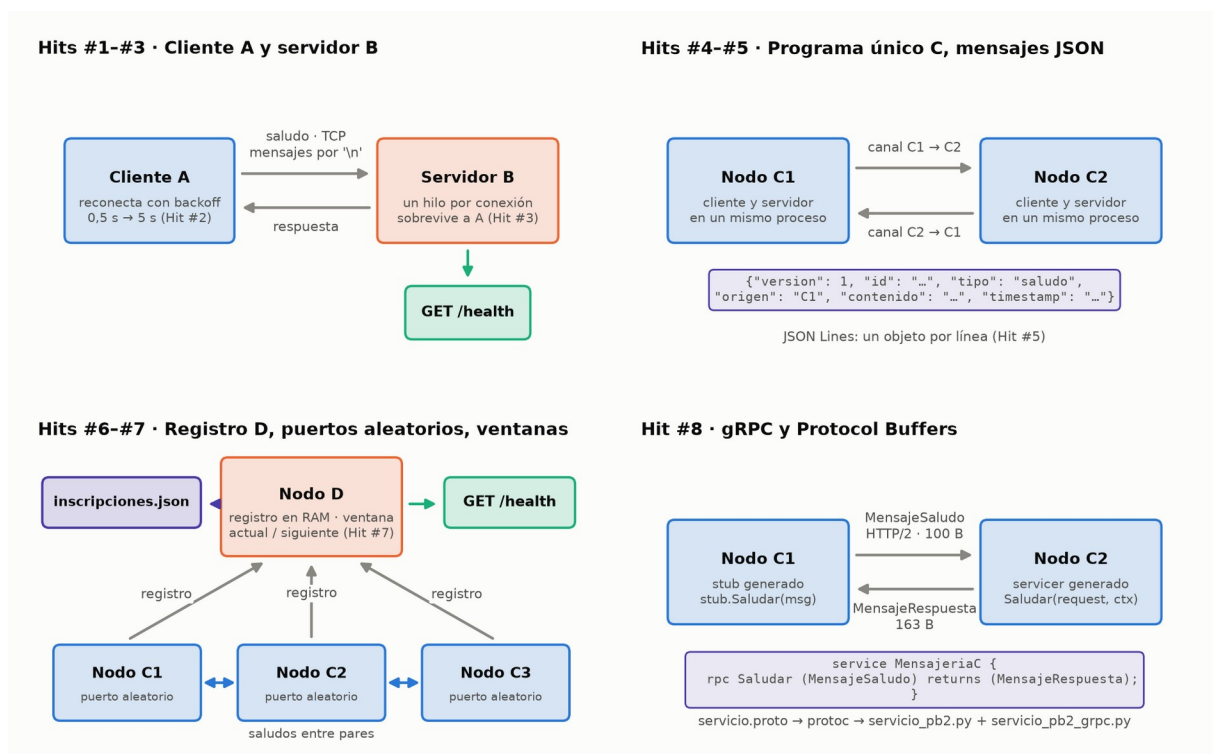


Figura 1. Evolución de la arquitectura a lo largo de los ocho hits.

## El módulo común

Cinco piezas compartidas por todos los hits, cada una con una sola responsabilidad:

| Módulo             | Responsabilidad                                    | Decisión clave                                                                                                                                                                         |
|--------------------|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| comun/protocolo.py | Delimitación de mensajes sobre el flujo TCP.       | Buffer hasta '\n', tope de tamaño, y todo fallo de lectura traducido a la familia ErrorDeProtocolo (un UnicodeDecodeError crudo hereda de ValueError y escapaba a los except OSError). |
| comun/mensajes.py  | Serialización JSON y validación (desde el Hit #5). | Compacto (sin espacios), UTF-8 sin escapar, campos obligatorios verificados al deserializar.                                                                                           |
| comun/health.py    | Endpoint HTTP GET /health con JSON.                | Hilo daemon aparte; escucha en la misma interfaz que el servicio; si el puerto está ocupado el servicio arranca igual.                                                                 |
| comun/registro.py  | Logs en tres destinos.                             | Consola + buffer circular en RAM (500 registros) + archivo rotativo en logs/ (1 MB, 3 copias).                                                                                         |
| comun/config.py    | Configuración por entorno.                         | Precedencia argumento CLI > variable de entorno > default; lector de .env propio, sin dependencias.                                                                                    |

## Patrones que se repiten

- **Ningún hilo muere en silencio.** Un traceback en stderr no pasa por el logger y no queda en logs/; por eso cada hilo cierra con un except de último recurso que registra y sigue.
- **Fallar sin tumbar al otro.** Un kill -9 llega como RST; bytes ilegibles, un JSON malformado o un mensaje de tipo desconocido se cuentan en el /health y se descartan, nunca cortan el canal.
- **Esperas interrumpibles.** El backoff y la rotación de ventanas esperan sobre el evento de parada, no con sleep: detener() no queda bloqueado.
- **Observabilidad sin privilegio.** Todo servicio de larga vida expone /health con estado, uptime y contadores; publicarlo fuera de localhost es una decisión explícita (--host 0.0.0.0).

## 4. Métricas y gráficas: JSON sobre TCP frente a gRPC con Protobuf

El Hit #8 pide comparar los dos transportes en tres dimensiones. Las mediciones se hicieron con hit8/benchmarks.py sobre nodos C reales de los Hits #5 y #8, en loopback (Linux, Python 3.14, grpcio 1.83.1).

### 4.1 Tamaño de los mensajes

Se serializa el mismo mensaje lógico (mismos UUID, origen, contenido y timestamp) con comun.mensajes.serializar (JSON compacto, tal como viaja en el Hit #5) y con SerializeToString() de Protobuf. Se cuentan sólo los bytes del mensaje, sin el framing de transporte.

| Mensaje      | JSON (UTF-8) | Protobuf (binario) | Reducción | Datos útiles | Sobrecarga JSON | Sobrecarga Protobuf |
|--------------|--------------|--------------------|-----------|--------------|-----------------|---------------------|
| Saludo       | 161 B        | 100 B              | -37,9 %   | 89 B         | 72 B            | 11 B                |
| Respuesta    | 242 B        | 163 B              | -32,6 %   | 150 B        | 92 B            | 13 B                |
| Ida y vuelta | 403 B        | 263 B              | -34,7 %   | 239 B        | 164 B           | 24 B                |

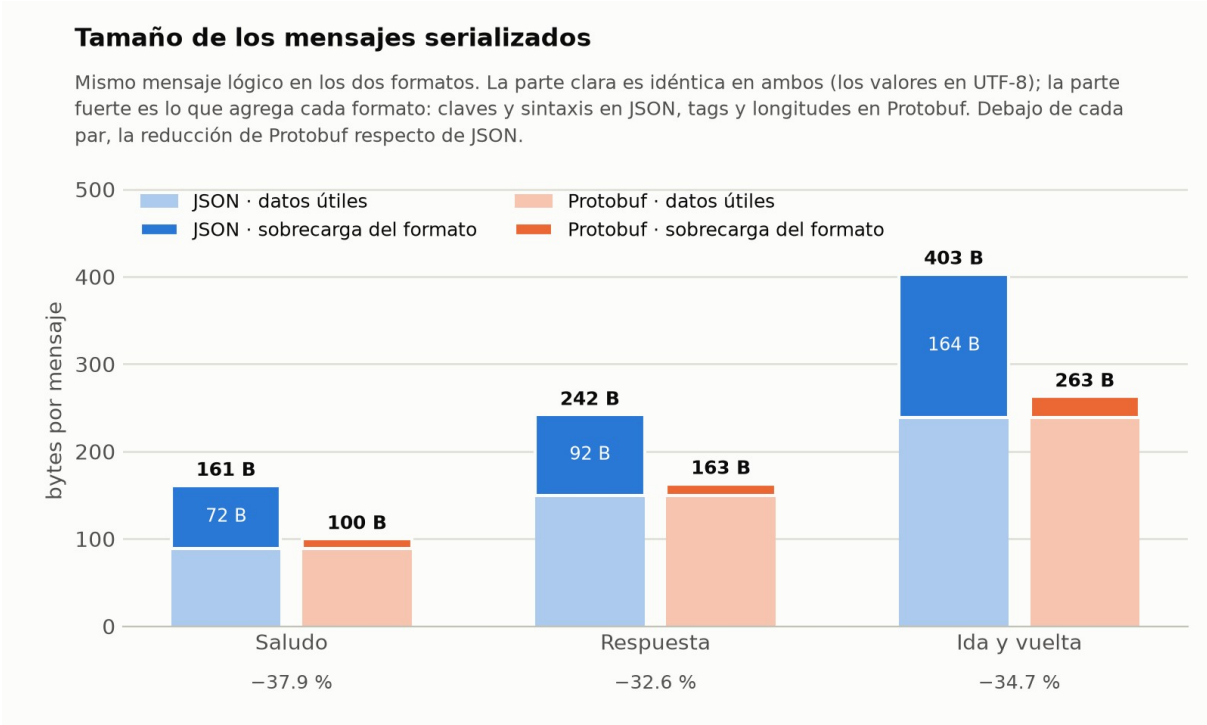


Figura 2. Tamaño de los mensajes serializados. La parte clara (los valores en UTF-8) es idéntica en los dos formatos; la parte fuerte es lo que agrega cada formato.

Los datos útiles —los valores de cada campo codificados en UTF-8— pesan lo mismo en ambos formatos. La diferencia está en la sobrecarga: JSON transmite en cada mensaje los nombres de las claves y la sintaxis (llaves, comillas, comas, dos puntos), 72 bytes en el saludo; Protobuf reemplaza cada nombre por un tag numérico de un byte más un byte de longitud, 11 bytes. Como los UUID y el timestamp ISO 8601 son cadenas largas que viajan igual en los dos, la reducción queda en torno al 35 %; con más campos numéricos o cadenas cortas, la ventaja de Protobuf sería mayor.

## 4.2 Latencia de las llamadas

1000 llamadas ida y vuelta consecutivas por transporte, sobre una conexión ya establecida; las 20 primeras, de calentamiento, no se cuentan.

| Métrica              | JSON sobre socket TCP persistente    | gRPC sobre HTTP/2 (Protobuf)              |
|----------------------|--------------------------------------|-------------------------------------------|
| Mediana (p50)        | 0,052 ms                             | 0,320 ms                                  |
| p95                  | 0,062 ms                             | 0,366 ms                                  |
| p99                  | 0,106 ms                             | 0,399 ms                                  |
| Promedio             | 0,054 ms                             | 0,319 ms                                  |
| Mínima / máxima      | 0,051 ms / 0,143 ms                  | 0,262 ms / 0,431 ms                       |
| Framing / transporte | Socket L4 directo, mensajes por '\n' | Frames HTTP/2 + prefijo gRPC + ThreadPool |

### Latencia ida y vuelta por llamada

1000 llamadas consecutivas por transporte, en loopback, sobre una conexión ya abierta (las 20 primeras, de calentamiento, no se miden). Valores en milisegundos.

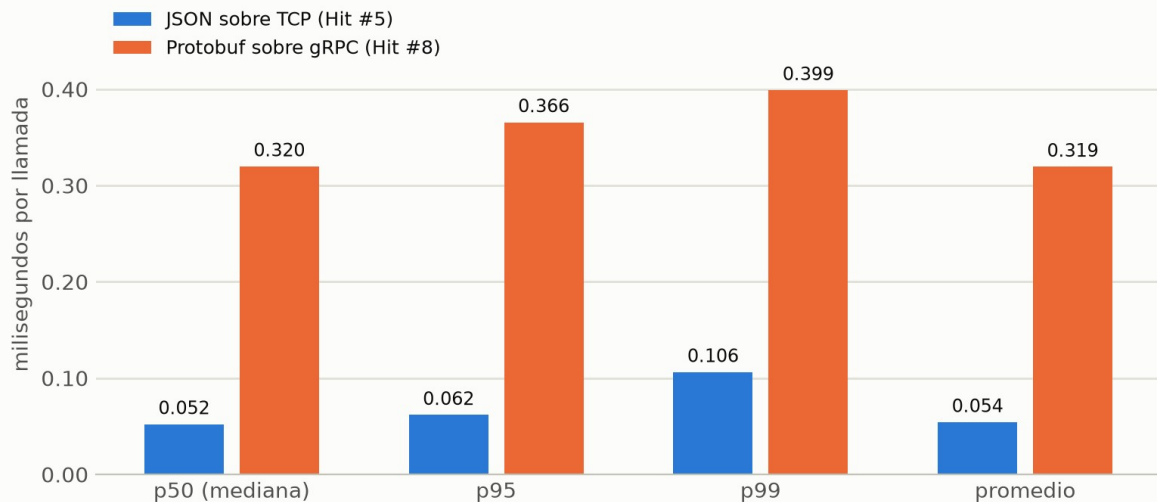


Figura 3. Latencia ida y vuelta por llamada: percentiles y promedio.

### Distribución acumulada de la latencia

Cada curva sube donde se concentran las llamadas: cuanto más a la izquierda y más vertical, más rápido y más estable es el transporte. El punto marca la mediana.

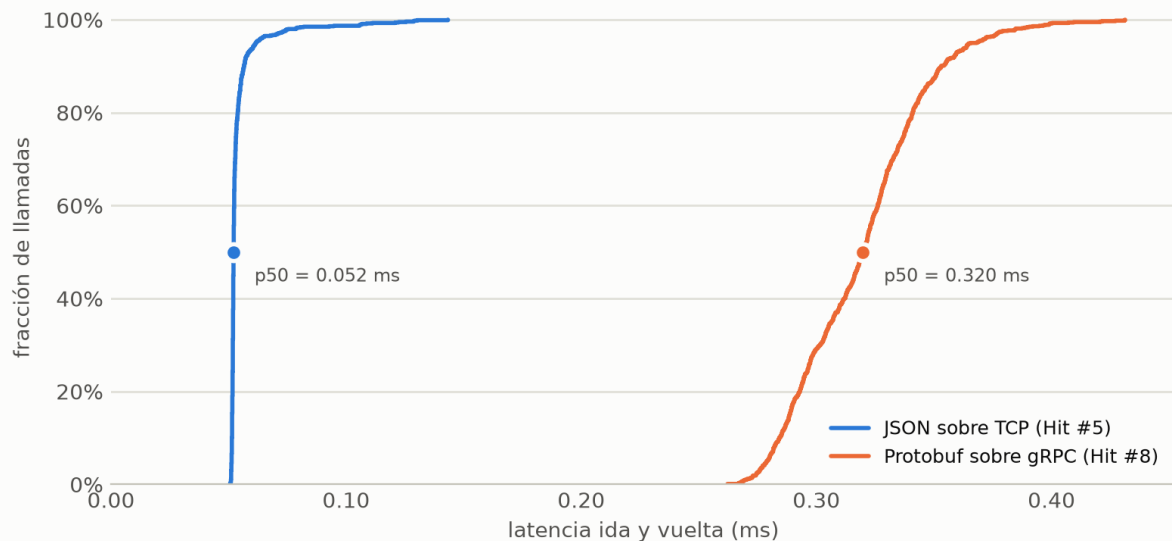


Figura 4. Distribución acumulada de la latencia; el punto marca la mediana.

En loopback, el socket crudo con un parser elemental es unas seis veces más rápido: no hay casi capas intermedias. gRPC paga la máquina de estados de HTTP/2 (cabeceras HPACK, streams lógicos) y las capas de contexto e interceptores de Python. Pero la diferencia es un costo fijo de ~0,27 ms por llamada: en una red real, donde el solo viaje de ida y vuelta entre dos máquinas cuesta entre 0,5 ms (LAN) y decenas de milisegundos (Internet), queda diluida. La distribución acumulada muestra además que los dos transportes son estables: el 95 % de las llamadas cae dentro de 0,01 ms (JSON) y 0,05 ms (gRPC) de la mediana. En redes reales y con tráfico concurrente, la multiplexación de HTTP/2 sobre una única conexión, el streaming bidireccional y el menor ancho de banda de Protobuf pesan más que ese costo fijo.

## 4.3 Experiencia de desarrollo

| Aspecto              | Hit #5 — JSON manual sobre TCP                                                           | Hit #8 — gRPC / Protobuf                                                                                 |
|----------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Contrato             | Implícito en diccionarios y en la documentación; riesgo de desalineación entre nodos.    | Explícito y estricto en servicio.proto (IDL independiente del lenguaje).                                 |
| Validación de tipos  | Manual: isinstance, claves obligatorias, manejo defensivo de KeyError / JSONDecodeError. | Fuerte, en los stubs generados; un campo de tipo incorrecto no compila.                                  |
| Transporte y framing | Artesanal: buffers, delimitador, recv en trozos, timeouts.                               | Transparente: stub.Saludar(mensaje).                                                                     |
| Errores              | Excepciones propias sobre el socket, traducción manual de desconexiones.                 | Códigos estándar (UNAVAILABLE, INVALID_ARGUMENT, DEADLINE_EXCEEDED) y RpcError.                          |
| Interoperabilidad    | Cada lenguaje cliente parsea el JSON a mano.                                             | protoc genera clientes en C++, Java, Go, Rust, TypeScript, etc.                                          |
| Costo                | Sin dependencias; legible con tcpdump o nc.                                              | Dos dependencias (grpcio, grpcio-tools), un paso de generación y mensajes binarios que no se leen a ojo. |

En síntesis: el Hit #5 costó más código y más pruebas de casos borde (cada uno de los fallos de framing y validación se descubrió y se fijó a mano); el Hit #8 costó entender la herramienta, y a cambio el contrato quedó escrito una sola vez y el código de transporte desapareció.

## 5. Pruebas

82 pruebas con unittest de la biblioteca estándar, todas ejecutables desde la raíz con `python -m unittest discover -s . -t . -v`. Cada hit prueba lo que ese hit agrega, sobre sockets reales en puertos efímeros (integración) o sobre funciones puras (unitarias).

| Módulo     | Pruebas | Qué cubren                                                                                                                                                                               |
|------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| comun      | 21      | Framing por '\n' (mensajes partidos, pegados, demasiado largos, ilegibles), formato JSON, /health, logs en memoria y disco, configuración por entorno.                                   |
| hit1       | 2       | Construcción de la respuesta; intercambio A ↔ B sobre un socket real.                                                                                                                    |
| hit2       | 8       | Backoff y su tope; B muere y vuelve; B nunca disponible; B callado (sin reconexión por timeout); A muere de golpe (RST); línea ilegible descartada.                                      |
| hit3       | 6       | Corte abrupto del cliente; conexiones sucesivas y concurrentes; bytes ilegibles; cliente mudo cerrado por inactividad; el /health refleja las conexiones.                                |
| hit4       | 7       | Saludo mutuo; canales independientes; par inexistente y aparición tardía; caída del par; recuperación ante basura; la lista de hilos no crece.                                           |
| hit5       | 16      | Formato JSON (obligatorios, unicidad de id, correlación, acentos, una sola línea); JSON crudo en el socket; mensajes inválidos; sólo se responde a saludos; dos mensajes en un segmento. |
| hit6       | 5       | Mensajes de registro; N nodos C con puertos aleatorios distintos descubiertos vía D; /health de D.                                                                                       |
| hit7       | 5       | Registro para la ventana siguiente; visibilidad sólo de la actual; rotación; persistencia JSON.                                                                                          |
| hit8       | 8       | Serialización Protobuf y tamaño; comunicación gRPC entre nodos; un solo saludo por conexión; RpcError ante mensajes inválidos; registro y descubrimiento contra el Nodo D gRPC.          |
| despliegue | 4       | El arranque del contenedor levanta D y los nodos C y el apagado por SIGTERM es limpio; el HEALTHCHECK falla si nadie escucha; el nombre del Nodo D lleva el commit desplegado.           |

A las pruebas se suma la prueba de humo end-to-end del pipeline, que levanta los procesos de verdad: A saluda a B; se mata a A con `kill -9` y B sigue respondiendo el health; dos C se saludan; los mensajes viajan en JSON; tres C se descubren a través de D; las inscripciones del Hit #7 quedan en disco; los nodos del Hit #8 se comunican por gRPC; y la imagen Docker del despliegue arranca con sus tres nodos C registrados.

## 6. Integración continua y despliegue público

### 6.1 El pipeline

GitHub Actions, en cada push y pull request a main o dev, en este orden (publicar y desplegar, sólo desde main):

1. **gitleaks** — falla si detecta un secret hardcodeado en el historial.
2. **Pruebas** — comun, los ocho hits y el arranque del despliegue, en Python 3.11, 3.12 y 3.13.
3. **Prueba de humo** — los procesos reales de cada hit, verificados por su /health o por el socket.
4. **Imagen Docker** — construye la imagen con el commit horneado, la levanta, espera a que los tres C estén registrados en D y comprueba que el proceso no corra como root. Sólo en push a main, y sólo si todo eso pasó, la publica en el registro de GitHub (GHCR) como ghcr.io/mnomico/sdypp\_tp1, con las etiquetas :<sha> y :latest.
5. **Despliegue** — sólo en push a main y con todo lo anterior en verde. El pipeline no entra a la VM: comprueba que la imagen se pueda traer sin credenciales y espera, hasta 10 minutos, a que el /health público responda con el SHA del commit recién pusheado. Recién entonces da el job por exitoso.

### 6.2 El despliegue

**URL pública del health-check:** `http://18.231.127.74:8080/health`

El servicio corre en una máquina virtual t3.micro de AWS EC2, en la región de São Paulo (sa-east-1), con Ubuntu y Docker. Es HTTP plano sobre la IP pública de la instancia: no hay dominio ni HTTPS (ver Decisiones).

Se despliega una sola imagen Docker con todo el repositorio. Por defecto arranca el Nodo D del Hit #6 (registro de contactos) con su /health en el puerto 8080, y tres nodos C que se registran contra él dentro del mismo contenedor: el health muestra un registro real, con nodos, puertos aleatorios asignados por el sistema operativo, uptime y conexiones atendidas. Son los mismos python -m hit6.nodo\_d y python -m hit6.nodo\_c del README, supervisados por un proceso padre (despliegue/arranque.py) que reenvía SIGTERM y termina si D muere.

La VM no recibe órdenes de nadie. Se preparó una sola vez, a mano: Docker, el docker-compose.yml del repositorio en /opt/sdypp-tp1 y un timer de systemd (sdypp-tp1.timer) que cada minuto ejecuta docker compose pull y docker compose up -d. Si el pipeline publicó una imagen :latest nueva, la trae y reemplaza el contenedor; si no, no hace nada. El nombre del Nodo D lleva el commit que trae la imagen (NodoD-nube@<sha>), y eso es lo que el pipeline mira en el /health para saber que la versión nueva ya está sirviendo.

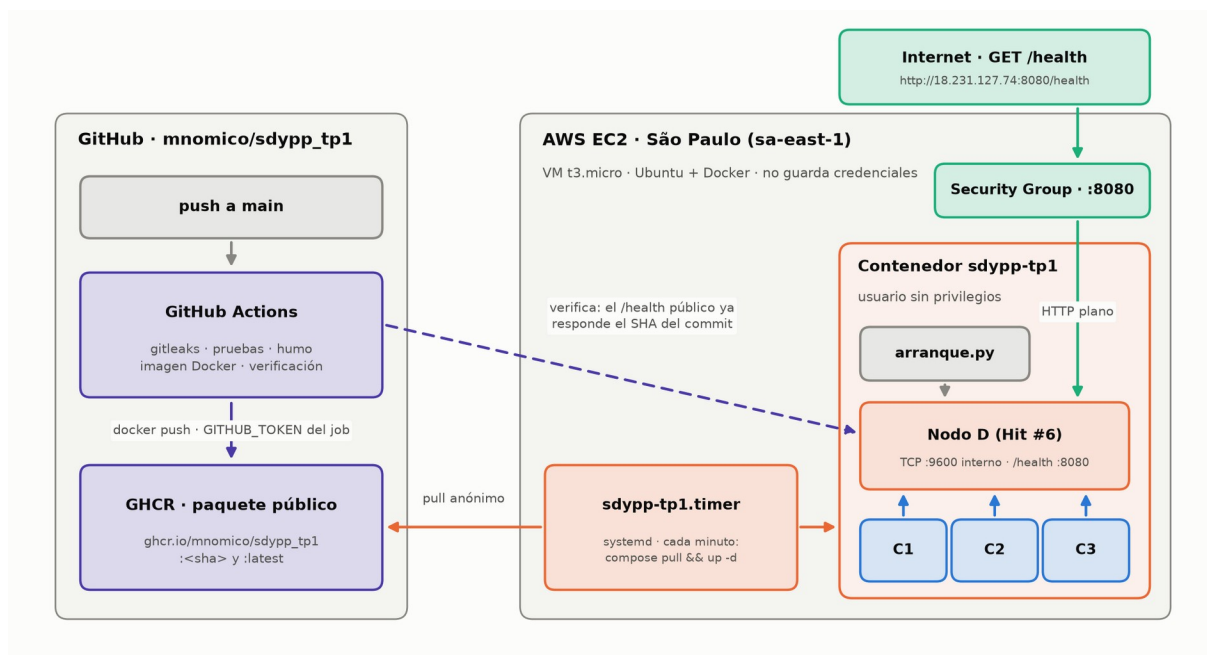


Figura 5. Del push a main al servicio público: el pipeline publica la imagen en GHCR y la VM de AWS EC2 la trae por su cuenta.

```
$ curl -s http://18.231.127.74:8080/health | python -m json.tool
{
  "servicio": "hit6-nodo-d",
  "nombre": "NodoD-nube@9e7542f",
  "estado": "ok",
  "estado_general": "ok",
  "uptime": 1953591.099,
  "uptime_segundos": 1953591.099,
  "escuchando_en": "0.0.0.0:9600",
  "cantidad_nodos_c_registrados": 3,
  "nodos_c_registrados": 3,
  "nodos": [
    {"ip": "127.0.0.1", "puerto": 45647, "nombre": "C1"},
    {"ip": "127.0.0.1", "puerto": 39455, "nombre": "C2"},
    {"ip": "127.0.0.1", "puerto": 37513, "nombre": "C3"}
  ],
  "conexiones_activas": 0,
  "conexiones_atendidas": 3,
  "mensajes_invalidos": 0
}
```

Salida real del 06/10/2026, antes de publicar esta revisión (cada nodo abreviado a una línea). El nombre dice qué commit está sirviendo, y el uptime —22,6 días— que el contenedor corría sin interrupciones desde el despliegue del 14/09.

## Cómo se opera

Si el health no responde, el orden para mirar es: el pipeline (¿el último push a main publicó la imagen?), el paquete en GHCR (¿sigue siendo público?) y la VM. En la VM se entra por SSH con la clave del grupo, que no está en el repositorio:

```
systemctl list-timers sdyp1-tp1.timer      # ¿corre el timer? cuándo es la próxima actualización
journalctl -u sdyp1-tp1.service -n 20     # qué hizo el último pull / up
docker ps; docker logs sdyp1-tp1 --tail 40 # el contenedor y sus registros
curl -s http://127.0.0.1:8080/health      # el health, desde adentro de la VM
```

Si la IP de la instancia cambia, se carga la variable de repositorio TP1\_URL para que el pipeline verifique la URL nueva y se actualizan los README.

## Decisiones

- **Una VM de AWS y no Cloud Run.** El plan original, y lo que decía la versión anterior de este informe, era Google Cloud Run. No pudimos habilitar la facturación de la cuenta de GCP y lo descartamos; Render, la alternativa gratuita que probamos, apaga el servicio cuando no recibe tráfico, y acá los procesos tienen que seguir vivos entre un request y otro. Quedó una VM t3.micro del plan gratuito de AWS. Se gana un host que controlamos entero (procesos de larga vida, puertos TCP, systemd) y que reutilizamos en el TP2. Se paga con un sistema operativo que hay que mantener, una única máquina sin redundancia, y sin el HTTPS y el dominio que Cloud Run daba resueltos.
- **Despliegue por pull, no por push.** En un despliegue por push el pipeline entra a la máquina (por SSH o con la API del proveedor) y ejecuta el despliegue: para eso GitHub tiene que guardar una clave SSH o credenciales de AWS, y la VM tiene que aceptar esas conexiones entrantes. Acá el sentido es el inverso: el pipeline sólo publica la imagen y es la VM la que pregunta cada minuto si hay una nueva. Nadie de afuera puede ordenarle qué correr: sólo puede traer lo que el pipeline publicó, y el pipeline publica únicamente desde main y con gitleaks, las pruebas, el humo y la prueba de la imagen en verde. Es el esquema de reconciliación de Kubernetes: se declara el estado deseado (la imagen :latest) y un agente local converge hacia él. El costo: el despliegue tarda hasta un minuto más; el pipeline no puede forzarlo, sólo esperar y verificar el SHA en el /health; y como :latest es una etiqueta mutable, en el flujo automático volver atrás es publicar de nuevo (un revert en main), no elegir una versión anterior.
- **Sin secrets.** El repositorio no tiene ningún secret cargado a mano. Para publicar, el pipeline usa el GITHUB\_TOKEN que GitHub emite para cada job y que vence cuando el job termina, con permiso de escritura de paquetes sólo en el job que publica. Para traer la imagen la VM no usa nada: el paquete de GHCR es público y el pull es anónimo; como el repositorio también es público, la imagen no expone nada que no esté ya en el código. Y no hay credenciales de AWS en ningún lado, porque nada automatizado habla con la API de AWS. Una credencial que no existe no se filtra, no se rota y no se revoca. No es OIDC ni Workload Identity Federation: no hay federación de identidad porque el pipeline no se autentica contra el proveedor de nube. La única credencial de larga vida del sistema es la clave SSH de administración de la VM, que queda fuera del repositorio y de GitHub.
- **HTTP plano sobre una IP: qué se pierde.** El TP1 pide un endpoint público que devuelva JSON y no pide HTTPS. Sin TLS se pierden tres cosas: confidencialidad (cualquiera en el camino puede leer la respuesta), integridad (un intermediario puede alterarla sin que el cliente lo note) y autenticidad (sin certificado, el cliente no puede comprobar que le responde nuestro servidor). Y sin dominio, la URL es la IP pública de la instancia: si cambia la IP, cambia la URL. Para este health lo aceptamos: es de sólo lectura y no viajan credenciales ni datos personales. No sería aceptable para un endpoint que reciba escrituras o credenciales, y es una de las razones por las que el puerto de registro no está publicado. Lo resolvemos en el TP3, con dominio y certificado TLS delante del servicio.
- **Un puerto publicado, el del /health.** El puerto TCP de registro (9600) queda interno al contenedor y los nodos C corren como procesos hermanos. Para que nodos C de otras máquinas se anoten en D alcanza con publicar el 9600 en docker-compose.yml y abrirlo en el Security Group; se dejó cerrado porque la consigna pide público el health, no el registro.
- **Una sola instancia, estado efímero.** Hay una VM y un contenedor: el /health muestra siempre el mismo Nodo D, pero no hay alta disponibilidad. Cada despliegue reemplaza el contenedor (unos segundos sin servicio, y el registro, que vive en RAM por consigna, arranca vacío hasta que los C se vuelven a anotar), y si la VM se cae, se cae el servicio. restart: unless-stopped y el timer lo levantan de nuevo cuando la VM vuelve. Los logs en disco viven en el contenedor y se pierden con cada despliegue.

- **Imagen mínima.** Multi-stage, base python:3.13-slim versionada, usuario sin privilegios, HEALTHCHECK, sin secrets en ENV, .dockerignore que deja afuera .git, .venv, logs/ y .env. Pesa 64 MB comprimida.

## 7. Seguridad

- No se versionan credenciales ni archivos .env: sólo la plantilla .env.example. .gitignore desde el primer commit.
- No hay direcciones, puertos ni secrets hardcodeados: todo sale de variables de entorno con defaults de desarrollo.
- gitleaks corre en cada push y hace fallar el pipeline si detecta un secret.
- El despliegue no usa ninguna credencial guardada: el pipeline publica la imagen con el GITHUB\_TOKEN efímero del job y la VM la trae de un paquete público con un pull anónimo. No hay clave SSH ni credenciales de AWS en GitHub, ni credenciales de GitHub en la VM. No es OIDC / Workload Identity Federation, porque el pipeline no se autentica contra el proveedor de nube; ése es el mecanismo que corresponde cuando tenga que hacerlo (TP3).
- El contenedor corre con un usuario sin privilegios (el pipeline comprueba que no sea root) y sin volúmenes ni el socket de Docker del host montados.
- El contenedor sólo publica el puerto del /health (8080); el puerto de registro (9600) queda interno.
- El /health público viaja por HTTP sin cifrar: no hay confidencialidad ni integridad en tránsito. Es un riesgo aceptado para un endpoint de sólo lectura sin datos sensibles; HTTPS queda para el TP3 (sección 6).
- Los /health escuchan en la misma interfaz que el servicio: publicar uno es una decisión explícita, no un default.
- Si un secret se filtrara, se revoca de inmediato: queda en el historial de Git aunque se borre del árbol.

## 8. Herramientas de IA

Usamos dos asistentes.

### Claude (Anthropic), a través de Claude Code en la terminal

Lo usó Tomás para realizar los hit 1-2-3-4-5. Ayudó a:

- Documentar (Readme).
- Aprender gRPC y como funciona (tambien entender codigo hecho por mi equipo).
- Syntaxis.
- Github action con test automaticos.
- Corrección de codigo.

Lo usó Mateo en el Hit #8. Ayudó a:

- Definir el archivo .proto a partir de los mensajes JSON del Hit #5 (mismos campos, ahora tipados).
- Entender la generación de stubs con grpc\_tools.protoc y la estructura de un servicer.
- Migrar el código de sockets a las llamadas gRPC generadas y traducir los errores a RpcError.
- Armar el script de benchmark y la primera versión de la comparativa de tamaño, latencia y experiencia de desarrollo.

## Google Gemini (Google), a través de Antigravity IDE

Lo usó Salvador en los Hits #6 y #7. Ayudó a:

- Diseñar e implementar el Nodo D como servicio de registro centralizado y descubrimiento dinámico de Nodos C con asignación de puertos aleatorios (Hit #6).
- Desarrollar el sistema de inscripciones por ventanas de tiempo de 1 minuto, coordinando el hilo daemon de rotación y la persistencia atómica en JSON (Hit #7).
- Definir la estructura de mensajes en formato JSON y el manejo del flujo TCP para registro, consultas de activos y respuestas.
- Implementar las suites de pruebas unitarias e integración con `unittest` para validar la concurrencia (`threading.Lock`), reconexiones y resiliencia.
- Redactar la documentación técnica (`README.md`), generar los diagramas de secuencia Mermaid y organizar la reutilización de módulos en `comun/`.

## 9. Conclusiones

- Un send no se corresponde con un recv. El framing por '\n' del Hit #1 sostuvo todo el sistema hasta el Hit #7, y el Hit #8 mostró qué se gana delegándolo a HTTP/2: el código de transporte desaparece.
- Un canal por sentido en el Hit #4 hizo que el orden de arranque no importara y que un nodo pudiera saludar a varios pares que nunca lo contactan (Hit #6). Separar serialización de transporte en el Hit #5 es lo que permitió cambiar JSON por Protobuf en el Hit #8 sin tocar los hits anteriores.
- Pasar de pegar cadenas a mensajes con version, id y en\_respuesta\_a cambió lo que el receptor entiende. campos, no texto. Validar al deserializar y descartar lo inválido sin cortar el canal es lo que permite convivir con nodos viejos o defectuosos.
- **RPC:** Un contrato escrito una sola vez, stubs generados, ~35 % menos bytes y errores tipados; a cambio, ~0,27 ms más por llamada en loopback, dos dependencias y mensajes que no se leen a ojo. Los casos de fallo de RPC (servidor caído, respuesta que no llega) siguen existiendo: gRPC los nombra (UNAVAILABLE, DEADLINE\_EXCEEDED), no los elimina.
- Health-checks en todos los servicios, logs en memoria y en disco, configuración por entorno, 82 pruebas más una prueba de humo y una imagen probada en cada push, y un despliegue sin claves.

## 10. Video y anexo: cómo ejecutar

### Video

Hay un video por hit, con ese hit corriendo, embebido en la sección Demo del README de cada carpeta (hit1/README.md a hit8/README.md). Se reproducen directamente en GitHub, sin descargar nada.

## Ejecución desde la terminal

Todo se ejecuta desde la raíz del repositorio, sin IDE. Cada programa acepta --help.

```
git clone https://github.com/mnomico/sdypp_tp1 && cd sdypp_tp1
python -m venv .venv && source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt                             # sólo el Hit #8 necesita grpcio

python -m unittest discover -s . -t . -v                  # 82 pruebas

# Hit #6: un Nodo D y N nodos C con puerto aleatorio
python -m hit6.nodo_d --puerto 9600 --puerto-health 8086
python -m hit6.nodo_c --d-host 127.0.0.1 --d-puerto 9600 --nombre C1 --puerto-health 8081
curl http://127.0.0.1:8086/health

# Hit #8: gRPC con Protocol Buffers
python -m hit8.nodo_c --puerto 9501 --par-host 127.0.0.1 --par-puerto 9502 --nombre C1 --puerto-health 8501
python -m hit8.nodo_c --puerto 9502 --par-host 127.0.0.1 --par-puerto 9501 --nombre C2 --sin-health
python -m hit8.benchmarks                                  # la comparativa de la sección 4

# La imagen del despliegue, sin nube
docker build -t sdypp-tp1 . && docker run --rm -p 8080:8080 sdypp-tp1
curl http://127.0.0.1:8080/health
```

## Referencias

- Enunciado del TP1: <https://dpetrocelli.github.io/sd2026/practica-1.html>
- Repositorio: [https://github.com/mnomico/sdypp\\_tp1](https://github.com/mnomico/sdypp_tp1) (tag v1.1)
- Birrell, A. y Nelson, B. (1984). Implementing Remote Procedure Calls. ACM TOCS.
- Documentación de gRPC ([grpc.io](https://grpc.io)) y de Protocol Buffers ([protobuf.dev](https://protobuf.dev)).
- Despliegue: despliegue/README.md en el repositorio; documentación de Amazon EC2 ([docs.aws.amazon.com/ec2](https://docs.aws.amazon.com/ec2)) y de GitHub Container Registry ([docs.github.com/packages](https://docs.github.com/packages)).